

Interview Preparation -- AI Multi-Agent Code Review & Security Audit System

Table of Contents

1. [Project Elevator Pitch](#)
 2. [Problem Statement & Why I Built It](#)
 3. [High-Level Architecture](#)
 4. [Tech Stack & Why Each Choice](#)
 5. [The 4 AI Agents -- Deep Dive](#)
 6. [LangGraph Workflow & State Machine](#)
 7. [Code Walkthrough -- Module by Module](#)
 8. [Data Models \(Pydantic Schemas\)](#)
 9. [LLM Integration -- Dual Provider Architecture](#)
 10. [Pattern-Based Fallback Engine](#)
 11. [FastAPI Backend -- API Design](#)
 12. [Streamlit Frontend -- Dashboard](#)
 13. [Conditional Re-Audit \(Advanced Feature\)](#)
 14. [Security Considerations in the System Itself](#)
 15. [Sample Vulnerable App \(Demo Data\)](#)
 16. [Deployment \(Render\)](#)
 17. [Challenges Faced & How I Solved Them](#)
 18. [Potential Interview Questions & Answers](#)
 19. [Future Enhancements](#)
 20. [Key Terminology Glossary](#)
-

1. Elevator Pitch

"I built a **Multi-Agent AI system** that automatically analyzes any codebase for **security vulnerabilities** and **code quality issues**. A user uploads a ZIP file or pastes a GitHub URL, and four AI agents -- orchestrated through a **LangGraph state machine** -- sequentially scan the code, detect **OWASP Top 10** vulnerabilities like SQL Injection and XSS, review code quality for anti-patterns and performance issues, and generate a comprehensive audit report with a **health score from 0-100**. The entire pipeline has a fallback pattern-based scanner so it works even without an LLM, and includes a **conditional re-audit edge** that triggers deeper analysis when too many critical vulnerabilities are found."

2. Problem Statement

What problem does this solve?

- * Manual code review is time-consuming and inconsistent -- human reviewers miss things depending on fatigue, expertise, and time pressure.
- * Security vulnerabilities (SQL injection, XSS, hardcoded secrets) are common in codebases but often slip through regular code reviews.
- * Existing tools like SonarQube are heavy, require infrastructure setup, and aren't AI-powered.
- * There's no lightweight tool that combines security auditing + code quality analysis + LLM-powered intelligence in one pipeline.

My solution:

An AI-powered system where multiple specialized agents each handle one domain (scanning, security, quality, reporting), orchestrated by a **state machine** (not just a chain), giving us conditional branching logic and clear state transitions.

3. High-Level Architecture

4. Tech Stack & Justifications

Component	Technology	Why I Chose It
Backend Framework	FastAPI	Async-native, auto-generates OpenAP..
Agent Orchestration	LangGraph	Proper **state machine** with condi..
LLM (Local)	Ollama + Mistral	Free, runs locally, keeps code priv..
LLM (Cloud)	OpenAI / Groq	Better results for complex analysis..
Frontend	Streamlit	Rapid prototyping, built-in widgets..
Charts	Plotly	Interactive charts, gauge charts fo..
Data Validation	Pydantic v2	Type-safe schemas, automatic valida..
File Parsing	Python `ast` module	Parse Python files into ASTs to det..
Config	python-dotenv	Load environment variables from `.e..
Git Operations	GitPython	Clone repos programmatically for th..
Deployment	Render	Free tier, supports both web servic..

Why LangGraph over CrewAI?

This is a common interview question. Key reasons:

- * State Machine Control: LangGraph uses StateGraph -- I define nodes, edges, and conditional edges explicitly. CrewAI is more of an agent-chain with less control.
- * Conditional Routing: My re-audit feature requires a conditional edge (if critical_findings > 3, route to re_audit). LangGraph supports this natively with add_conditional_edges().
- * Typed State: LangGraph uses TypedDict for state, so every agent reads/writes to a shared, typed state object.
- * Deterministic Flow: The pipeline always follows the same sequence (scanner -> security -> code review -> report), with one conditional branch. This is more like a workflow than autonomous agents deciding what to do next.

5. The 4 AI Agents

Agent 1: Scanner Agent (`scanner.py`)

Purpose: Traverse the file tree and extract relevant source code files for analysis.

What it does:

- * Walks the directory tree using os.walk()
- * Filters files by supported extensions (.py, .js, .ts, .java, .go, .rb)
- * Skips directories like node_modules/, venv/, .git/, __pycache__/, dist/, build/
- * Skips files larger than 100KB (likely auto-generated)
- * For each file:
- * Detects the programming language from the extension
- * Extracts imports (using Python's ast module for .py files, regex for JS/TS)
- * Determines if it's an entry point (checks filename patterns like main.py, app.py, or presence of route decorators like @app.route)
- * Sorts files with entry points first (prioritized for analysis)
- * Updates state: files list + status -> "auditing"

Key implementation detail:

```
# Entry point detection checks both filename AND content
ENTRY_POINT_PATTERNS = {"main.py", "app.py", "server.py", "index.py", ...}
ROUTE_INDICATORS = {"@app.route", "@router.", "app.get", "app.post", ...}
```

State transition: `status: "scanning"` -> `status: "auditing"``

Agent 2: Security Auditor Agent (`security\_auditor.py`)

Purpose: Analyze code for OWASP Top 10 security vulnerabilities.

What it does:

- * Pattern-based scan first (always runs, no LLM needed):
- * Regex patterns for hardcoded secrets (API_KEY = "...", sk-..., AKIA...)
- * SQL injection patterns (string concatenation in execute(), f-strings in queries)
- * XSS patterns (unescaped user input in HTML responses)
- * Misc patterns: DEBUG=True, eval(), exec(), pickle.loads(), shell=True, directory traversal
- * Checks for admin routes without authentication decorators
- * LLM-enhanced analysis (if LLM is available):
- * Splits code into chunks of ~30,000 characters
- * Sends each chunk to the LLM with a structured prompt asking for OWASP Top 10 analysis
- * Parses JSON response and validates against SecurityFinding Pydantic model
- * Deduplicates findings by ID
- * Falls back gracefully if LLM fails or times out (120s timeout)

Vulnerability categories detected:

- * SQL Injection (string concatenation, f-strings, % formatting)
- * XSS (unescaped user input in HTML)
- * Hardcoded Secrets (API keys, passwords, tokens, AWS keys)
- * Broken Authentication (admin routes without auth)
- * Insecure Deserialization (pickle.loads, yaml.load)
- * Security Misconfiguration (DEBUG=True, CORS *, binding to 0.0.0.0)
- * Directory Traversal (user-controlled file paths)
- * Command Injection (os.system(), subprocess with shell=True)
- * Weak Cryptography (hashlib.md5, hashlib.sha1)

State transition: `status: "auditing"`` -> `status: "reviewing"``

Agent 3: Code Quality Reviewer Agent (`code\_reviewer.py`)

Purpose: Check code for anti-patterns, missing error handling, and quality issues.

What it does:

- * Pattern-based scan (always runs):
- * Bare except: clauses
- * Broad except Exception: catches
- * TODO / FIXME comments left in code

- * Wildcard imports (import *)
- * Global variable usage
- * time.sleep() in potentially async code
- * AST-based analysis (for Python files):
- * Function length check: Functions longer than 50 lines flagged as "god functions"
- * Nesting depth check: Code indented more than 4 levels deep
- * Missing type hints: Public functions (not prefixed with _) missing return type annotations
- * LLM-enhanced analysis (if available):
- * Similar chunking and prompting as the security auditor
- * Deduplicates findings by description to avoid overlap with pattern scan

State transition: `status: "reviewing"` -> `status: "generating"``

Agent 4: Report Generator Agent (`report\_generator.py`)

Purpose: Compile all findings into a final audit report.

What it does:

- * Counts severity distribution: Critical, High, Medium, Low
- * Calculates health score (0-100):
- * Starts at 100
- * Critical finding: -15 points
- * High finding: -10 points
- * Medium finding: -5 points
- * Low finding: -2 points
- * Quality issue: -2 points each
- * Clamped to range [0, 100]
- * Generates executive summary:
- * Default algorithmic summary (always works)
- * Optionally enhanced by LLM for more natural, professional language
- * Returns an AuditReport Pydantic model with all findings, scores, and summary

State transition: `status: "generating"`` -> `status: "complete"``

6. LangGraph Workflow

How the State Machine Works

```
# graph.py -- The core orchestration

class GraphState(TypedDict):
    source_path: str                # Input: directory path to scan
    files: list[dict]               # Scanner output: parsed files
    security_findings: list[dict]   # Security Auditor output
    quality_findings: list[dict]    # Code Reviewer output
    report: Optional[dict]          # Report Generator output
    status: str                     # Current pipeline stage
    error: Optional[str]            # Error message if pipeline fails

# Build the graph
workflow = StateGraph(GraphState)

# Add nodes (each is a function that takes state and returns partial state updates)
workflow.add_node("scanner", scanner_node)
workflow.add_node("security_auditor", security_auditor_node)
workflow.add_node("code_reviewer", code_reviewer_node)
workflow.add_node("re_audit", re_audit_node)
workflow.add_node("report_generator", report_generator_node)

# Set entry point
workflow.set_entry_point("scanner")

# Linear edges
workflow.add_edge("scanner", "security_auditor")
workflow.add_edge("security_auditor", "code_reviewer")

# CONDITIONAL EDGE -- the key differentiator
workflow.add_conditional_edges(
    "code_reviewer",
    should_re_audit,          # Decision function
    {
        "re_audit": "re_audit",
        "report_generator": "report_generator",
    },
)

workflow.add_edge("re_audit", "report_generator")
workflow.add_edge("report_generator", END)
```

Conditional Edge Logic

```
def should_re_audit(state: GraphState) -> str:
    critical_count = sum(
        1 for f in state["security_findings"] if f.get("severity") == "Critical"
    )
    if critical_count > 3:
        return "re_audit"          # Route to deeper analysis
    return "report_generator"      # Skip re-audit, go to report
```

Why this matters: This demonstrates that I understand **state machines** and **conditional routing** -- not just linear chains. The re-audit node does a focused deep-dive on only the files that had critical findings, looking for chained vulnerabilities and subtle issues the first pass missed.

How LangGraph State Updates Work

Each node function returns a **partial state dictionary**. LangGraph **merges** it into the existing state:

```
# scanner_node returns:
{"files": [...], "status": "auditing"}

# security_auditor_node returns:
{"security_findings": [...], "status": "reviewing"}

# These get merged into the shared GraphState automatically
```

Running the Pipeline

```
def run_audit(source_path: str) -> dict:
    initial_state = {
        "source_path": source_path,
        "files": [],
        "security_findings": [],
        "quality_findings": [],
        "report": None,
        "status": "scanning",
        "error": None,
    }
    result = audit_graph.invoke(initial_state)
    return result
```

7. Code Walkthrough -- Module by Module

Project Structure

```
code-audit-agent/
+-- main.py                # FastAPI app -- 4 REST endpoints
+-- app/
|   +-- config.py          # Settings class -- env vars, defaults
|   +-- models/
|   |   +-- schemas.py     # Pydantic models -- FileInfo, SecurityFinding, etc.
|   +-- agents/
|   |   +-- graph.py       # LangGraph StateGraph -- conditional edges
|   |   +-- scanner.py     # Agent 1 -- file tree traversal
|   |   +-- security_auditor.py # Agent 2 -- OWASP vulnerability detection
|   |   +-- code_reviewer.py # Agent 3 -- code quality analysis
|   |   +-- report_generator.py # Agent 4 -- report compilation
|   |   +-- pattern_scanner.py # Regex-based fallback scanner (no LLM)
|   |   +-- llm.py         # LLM factory -- Ollama/OpenAI/Groq/llama.cpp
|   +-- parsers/
|   |   +-- file_parser.py # File tree traversal, import extraction, AST parsing
|   +-- prompts/
|   |   +-- templates.py   # All LLM prompt templates (system + user)
|   +-- utils/
|   |   +-- helpers.py     # ZIP extraction, GitHub cloning, temp dir management
+-- frontend/
|   +-- streamlit_app.py   # Streamlit UI -- dashboard, charts, finding cards
+-- sample_repos/
|   +-- vulnerable_app/    # Intentionally vulnerable Flask app for demos
|   |   +-- app.py         # SQL injection, XSS, command injection, etc.
|   |   +-- auth.py        # Broken auth, MD5 hashing, plaintext passwords
|   |   +-- config.py      # Debug mode, hardcoded creds, no rate limiting
+-- render.yaml            # Deployment config for Render.com
```


Key File: `config.py`

```
class Settings:
    LLM_PROVIDER = os.getenv("LLM_PROVIDER", "ollama") # "ollama", "openai", or "pattern"
    OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
    OPENAI_BASE_URL = os.getenv("OPENAI_BASE_URL", "") # For Groq/Together/OpenRouter
    OPENAI_API_COMPAT = os.getenv("OPENAI_API_COMPAT", "standard") # "standard" or "llamacpp"
    SUPPORTED_EXTENSIONS = [".py", ".js", ".ts", ".java", ".go", ".rb"]
    SKIP_DIRS = {"node_modules", "venv", ".git", "__pycache__", "dist", "build", ...}
    MAX_FILE_SIZE_BYTES = 100 * 1024 # 100KB
```

All configuration is via environment variables -- **no hardcoded secrets** in the application itself.

Key File: `llm.py` -- LLM Provider Factory

This is the most architecturally interesting utility module:

- * Provider Detection: Checks if the configured LLM is reachable (health check)
- * Factory Pattern: `get_llm()` returns the correct LLM client based on config
- * Timeout Wrapper: `invoke_with_timeout()` uses `ThreadPoolExecutor` to enforce a 120-second hard timeout on LLM calls
- * Supports 4 backends:
 - * Ollama (local) via `langchain-ollama`
 - * OpenAI / Groq / Together via `langchain-openai`
 - * Llama.cpp server via a custom `CustomLlamaCppLLM` wrapper class
 - * Pattern-only mode (no LLM at all)

Key File: `pattern\_scanner.py` -- The Fallback Engine

This is the **reliability backbone** of the system. Even if the LLM is down, the system still works:

- * 40+ regex patterns for security vulnerabilities
- * 7 regex patterns for code quality issues
- * 3 AST-based checks for Python files (function length, nesting depth, type hints)
- * Used as a first-pass scanner that always runs, then LLM adds on top of pattern results

8. Data Models

Pydantic Schemas (`schemas.py`)

```

class FileInfo(BaseModel):
    path: str                # Relative path from root
    language: str            # "python", "javascript", etc.
    content: str             # Full file content
    is_entry_point: bool     # True if main.py, app.py, or has routes
    imports: list[str]       # Extracted import statements

class SecurityFinding(BaseModel):
    id: str                  # "SEC-001", "DEEP-001"
    severity: str            # "Critical", "High", "Medium", "Low"
    category: str            # "SQL Injection", "XSS", "Hardcoded Secret"
    file_path: str           # "app.py"
    line_number: Optional[int] # Line where vulnerability found
    code_snippet: str        # The actual vulnerable code
    description: str         # Explanation of the vulnerability
    fix_suggestion: str      # How to fix it
    confidence: float        # 0.0 to 1.0

class QualityFinding(BaseModel):
    id: str                  # "QUA-001"
    category: str            # "Anti-pattern", "Error Handling", "Performance"
    file_path: str
    description: str
    suggestion: str          # How to improve

class AuditReport(BaseModel):
    summary: str             # Executive summary text
    health_score: int        # 0-100
    total_critical: int
    total_high: int
    total_medium: int
    total_low: int
    security_findings: list[SecurityFinding]
    quality_findings: list[QualityFinding]
    files_analyzed: int

```

Why Pydantic?

- * Automatic validation -- if the LLM returns malformed data, Pydantic catches it
- * `.model_dump()` for easy serialization to JSON
- * Type safety throughout the codebase
- * Self-documenting field definitions

9. LLM Integration -- Dual Provider Architecture

How LLM Calls Work

- * Check availability: `is_llm_available()` pings the LLM endpoint
- * Get instance: `get_llm()` returns a LangChain-compatible LLM object
- * Build prompt: System prompt + user prompt with code context injected
- * Send with timeout: `invoke_with_timeout(llm, messages, timeout=120)`
- * Parse response: Extract JSON from LLM response (handles markdown code blocks, extra text)
- * Validate: Each finding is validated against Pydantic schemas
- * Deduplicate: Skip findings already found by pattern scanner

Robust JSON Parsing

The `_parse_findings_json()` function handles messy LLM output:

- * Strips markdown code block wrappers (`json ...`)
- * Tries direct `json.loads()` first
- * Falls back to regex-based JSON array extraction
- * Falls back to extracting individual JSON objects
- * Returns empty list if all parsing fails (never crashes)

Prompt Engineering

Each agent has a **system prompt** (defines the role and output format) and a **user prompt** (provides the code context):

- * Security Auditor: "You are an expert security auditor specializing in OWASP Top 10 vulnerabilities..."
- * Code Reviewer: "You are an expert code quality reviewer..."
- * Report Generator: "You are a technical report writer..."
- * Re-Audit: "You are a deep-dive security analyst... Look for chained vulnerabilities..."

All prompts request **JSON output matching the exact Pydantic schema** -- this ensures structured, parseable responses.

10. Pattern-Based Fallback Engine

This is a key architectural decision: **the system works without any LLM.**

Security Patterns (40+ regex patterns)

```
# Hardcoded secrets
r'(?:(API_KEY|SECRET_KEY|API_SECRET)\s*=\s*["\'](?:[^\s\']{8,}|["\']')
r'sk-[a-zA-Z0-9]{20,}' # OpenAI key pattern
r'AKIA[0-9A-Z]{16}' # AWS access key pattern

# SQL Injection
r'execute\s*\(\s*["\'].*?\s*\+\s*\w+' # String concatenation
r'execute\s*\(\s*f["\']' # F-string in SQL

# XSS
r'return\s+f["\']<.*?\{.*?request\.\w+' # Unescaped input in HTML

# Misc
r'pickle.loads?\s*\(' # Insecure deserialization
r'eval\s*\(' # Code injection
r'subprocess\.\.\.?shell\s*=\s*True' # Command injection
```

Quality Patterns (AST + Regex)

```
# Regex-based
r'except\s*:'          # Bare except clause
r'import\s+\*'         # Wildcard import
r'global\s+\w+'        # Global variable usage

# AST-based (Python-specific)
- Functions > 50 lines    # God function detection
- Nesting depth > 4 levels # Deep nesting
- Public functions missing return type hints
```

11. FastAPI Backend

API Endpoints

Method	Endpoint	Purpose
`GET`	`/`	Health check
`POST`	`/api/audit/upload`	Upload ZIP file -> start audit
`POST`	`/api/audit/github`	Clone repo URL -> start audit
`GET`	`/api/audit/{audit_id}/status`	Poll audit progress
`GET`	`/api/audit/{audit_id}/report`	Get final report

Async Execution Pattern

The audit pipeline runs in a **background thread** so the API responds immediately:

```
@app.post("/api/audit/upload")
async def upload_audit(file: UploadFile = File(...)):
    audit_id = str(uuid4())
    # ... extract zip ...
    audit_store[audit_id] = {"status": "queued", "report": None, ...}

    # Run audit in background thread (non-blocking)
    loop = asyncio.get_event_loop()
    loop.run_in_executor(None, _run_audit_pipeline, audit_id, source_path, temp_dir)

    return {"audit_id": audit_id, "status": "queued"} # Returns immediately
```

In-Memory Storage

```
audit_store: dict[str, dict] = {}
# Key: UUID audit_id
# Value: {"status": "scanning", "report": None, "error": None, "temp_dir": "/tmp/..."}
```

No database needed -- this is a stateless analysis tool. For production, you'd swap this with Redis or PostgreSQL.

Security in the API

- * ZIP path traversal protection: Validates no .. or absolute paths in ZIP entries
- * URL validation: Only HTTPS URLs accepted for GitHub cloning
- * Only GitHub/GitLab URLs: URL validation in `clone_github_repo()`

- * File size limits: Configured via MAX_FILE_SIZE_MB
 - * CORS middleware: Configured for web access
-

12. Streamlit Frontend

Layout

Sidebar:

- * Radio buttons: Upload ZIP / GitHub URL
- * File uploader or text input
- * "Start Audit" button
- * "How It Works" section

Main Area (during audit):

- * Progress bar with step indicators
- * Polls `/api/audit/{id}/status` every 2 seconds
- * Shows: Queued -> Scanning -> Security Audit -> Code Review -> Generating Report -> Complete

Main Area (after completion):

- * Top metrics row: Files Analyzed, Critical, High, Medium, Low counts
- * Health Score Gauge: Plotly gauge chart (green >80, yellow 50-80, red <50)
- * Severity Pie Chart: Plotly pie chart with Critical/High/Medium/Low distribution
- * Executive Summary: Info box with AI-generated summary
- * File Heatmap: Horizontal bar chart showing which files have the most findings
- * Security Findings: Expandable cards sorted by severity (Critical first)
- * Each card shows: severity badge, category, file path, description, line number, code snippet, confidence, fix suggestion
- * Quality Findings: Expandable cards with category, description, suggestion
- * Export: Download button for JSON report + "New Audit" button

Real-Time Progress Tracking

```
# Poll loop in Streamlit
for _ in range(300): # 10 minutes max
    status_data = poll_status(audit_id)
    current_status = status_data["status"]

    # Update progress bar
    step_idx = status_steps[current_status] # 0-5
    st.progress(step_idx / 5)

    if current_status == "complete":
        st.rerun()
        break

    time.sleep(2)
```

13. Conditional Re-Audit (Advanced Feature)

This is the **most impressive architectural feature** to highlight:

Trigger: If the Security Auditor finds > 3 Critical-severity vulnerabilities.

What it does:

- * Identifies the specific files that have critical findings
- * Sends only those files to the LLM with a deeper, more focused prompt
- * Looks for: chained vulnerabilities, logic flaws, race conditions, information disclosure
- * Uses a separate prompt template (RE_AUDIT_SYSTEM) that asks for deeper analysis
- * Merges new findings into the existing security findings list

Why this matters: It demonstrates understanding of:

- * Conditional routing in state machines
- * Adaptive analysis depth based on initial results
- * Focused re-processing (not re-scanning everything)

```
Normal flow:  Scanner -> Security -> Code Review -> Report
Re-audit flow: Scanner -> Security -> Code Review -> Re-Audit -> Report
```

14. Security Considerations in the System Itself

Even though this tool finds security issues in *other* code, the tool itself follows security best practices:

- * No hardcoded secrets: All API keys via env vars
- * ZIP path traversal protection: Validates archive entries before extraction
- * URL validation: Only HTTPS, only GitHub/GitLab domains
- * File size limits: Prevents resource exhaustion from large files
- * Temp directory cleanup: Auto-cleanup on shutdown via lifespan context manager
- * LLM timeout: 120-second hard timeout prevents hanging
- * Input validation: Pydantic models validate all API inputs
- * Repository isolation: Each audit gets its own temp directory
- * Shallow clones: depth=1 for GitHub repos reduces attack surface

15. Sample Vulnerable App

The ``sample_repos/vulnerable_app/`` contains **intentionally vulnerable** code for demos:

``app.py`` -- 10+ vulnerabilities:

- * SQL Injection via string concatenation and f-strings
- * XSS via unescaped user input in HTML
- * Command injection via `subprocess.run(cmd, shell=True)`
- * Insecure deserialization via `pickle.loads()`

- * Directory traversal via unsanitized `os.path.join()`
- * Hardcoded API key, DB password, AWS key
- * Debug mode enabled
- * Admin endpoints without authentication
- * Permissive CORS (*)
- * Binding to 0.0.0.0

``auth.py`` -- Broken authentication:

- * MD5 for password hashing
- * JWT with no verification (just base64 decode)
- * Plaintext password storage
- * No token expiration

``config.py`` -- Misconfigurations:

- * `DEBUG = True`
- * Hardcoded database URLs with passwords
- * `SESSION_COOKIE_SECURE = False`
- * `RATE_LIMIT_ENABLED = False`
- * Hardcoded encryption keys

When demonstrating, you zip this folder and upload it. The system should find **15-25+ vulnerabilities** across these files.

16. Deployment

Render Configuration (``render.yaml``)

Two services:

- * Backend (FastAPI): `uvicorn main:app --host 0.0.0.0 --port $PORT`
- * Frontend (Streamlit): `streamlit run frontend/streamlit_app.py --server.port $PORT`

The deployment uses **Groq** as the LLM provider (free tier, fast inference):

```
envVars:  
  - key: LLM_PROVIDER  
    value: openai  
  - key: OPENAI_MODEL  
    value: llama-3.3-70b-versatile  
  - key: OPENAI_BASE_URL  
    value: https://api.groq.com/openai/v1
```

17. Challenges Faced & Solutions

Challenge 1: LLM Response Parsing

Problem: LLMs return inconsistent JSON -- sometimes wrapped in markdown code blocks, sometimes with extra text before/after.

Solution: Built a robust `\_parse\_findings\_json()` function that tries multiple parsing strategies: direct parse -> regex array extraction -> individual object extraction -> empty fallback.

Challenge 2: LLM Unavailability

Problem: Ollama might not be running, API keys might be invalid, rate limits hit.

Solution: Built a pattern-based fallback engine with 40+ regex patterns. The system always produces results, just more basic without the LLM.

Challenge 3: Large Codebases

Problem: LLMs have token limits -- can't send a whole codebase in one prompt.

Solution: Chunk code into ~30,000 character blocks, limit to 3 chunks max, prioritize entry point files, send each chunk separately and merge results.

Challenge 4: Duplicate Findings

Problem: Pattern scanner and LLM might find the same issue.

Solution: Deduplicate by finding ID (security) and description (quality) before adding LLM results.

Challenge 5: LLM Timeouts

Problem: Some LLM calls take 60+ seconds, blocking the entire pipeline.

Solution: Used `concurrent.futures.ThreadPoolExecutor` with a 120-second hard timeout. If a chunk times out, it's skipped and the pipeline continues.

Challenge 6: Real-Time Progress

Problem: The audit takes 30-120 seconds -- user needs feedback.

Solution: Each agent updates `state["status"]`, the API exposes a status endpoint, and the Streamlit frontend polls every 2 seconds with a visual progress bar.

18. Potential Interview Questions & Answers

Q: "Walk me through what happens when a user uploads a ZIP file."

A:

- * User uploads ZIP via Streamlit -> HTTP POST to /api/audit/upload
- * FastAPI validates it's a ZIP, generates a UUID audit_id, creates a temp directory
- * ZIP is extracted with path traversal validation
- * Audit pipeline starts in a background thread via run_in_executor()
- * API returns {"audit_id": "...", "status": "queued"} immediately
- * Streamlit starts polling /api/audit/{id}/status every 2 seconds
- * Scanner Agent traverses the extracted directory, builds FileInfo list
- * Security Auditor runs pattern scan (40+ regex), then LLM analysis if available
- * Code Reviewer runs quality patterns + AST checks, then LLM if available
- * Conditional check: If > 3 Critical findings -> Re-Audit deep scan
- * Report Generator calculates health score, generates executive summary

* Status becomes "complete", Streamlit fetches and renders the report

Q: "Why did you use LangGraph instead of just calling functions in sequence?"

A: Three reasons:

- * Conditional routing: The re-audit edge can't be expressed in a simple function chain. LangGraph's `add_conditional_edges()` makes this clean and declarative.
 - * Shared typed state: All agents read/write to a `GraphState TypedDict`. LangGraph handles state merging automatically -- each node returns only the fields it updates.
 - * Extensibility: If I want to add a 5th agent (e.g., dependency vulnerability checker), I just add a node and an edge. The state machine pattern makes this trivial.
-

Q: "How do you handle the case where the LLM is not available?"

A: I built a **dual-layer analysis system**:

- * Layer 1 (always runs): Pattern-based scanner with 40+ regex patterns and Python AST analysis. This catches concrete patterns like hardcoded secrets, SQL injection via string concatenation, bare except clauses, etc.
- * Layer 2 (optional): LLM-enhanced analysis that catches contextual issues (e.g., logic flaws, missing auth on business-critical endpoints) that patterns can't detect.

The ``is_llm_available()`` function checks connectivity at startup. If the LLM is down, we log a warning and proceed with pattern-only results. The system **never crashes** due to LLM unavailability.

Q: "How does the health score work?"

A: It starts at 100 and deducts points:

- * Critical vulnerability: -15 points
- * High: -10
- * Medium: -5
- * Low: -2
- * Each quality issue: -2

Clamped to [0, 100]. For the sample vulnerable app with ~15 critical/high findings, the score drops to around 10-30, which correctly reflects severe issues.

Q: "What if the LLM returns malformed JSON?"

A: I built a 4-stage parsing pipeline in ``_parse_findings_json()``:

- * Try `json.loads()` directly
- * Strip markdown code blocks and try again
- * Regex-search for a JSON array anywhere in the text
- * Regex-extract individual JSON objects and collect valid ones

If all 4 stages fail, return an empty list -- the pattern-based findings are still preserved. Individual findings are also validated against Pydantic schemas with `try/except` -- malformed findings are logged and skipped.

Q: "How would you scale this to handle many concurrent users?"

A:

- * Replace in-memory store with Redis for audit state
 - * Use Celery/Redis Queue for background task processing instead of `run_in_executor`
 - * Add a database (PostgreSQL) for persistent audit history
 - * Containerize with Docker and use Kubernetes for horizontal scaling
 - * Rate limit API endpoints to prevent abuse
 - * Cache LLM results for identical code patterns
-

Q: "What OWASP Top 10 vulnerabilities does this detect?"

A:

- * A01 Broken Access Control -- Admin routes without auth decorators
 - * A02 Cryptographic Failures -- MD5/SHA1 hashing, hardcoded secrets
 - * A03 Injection -- SQL injection (3 patterns), command injection, XSS
 - * A04 Insecure Design -- Missing rate limiting, overly permissive CORS
 - * A05 Security Misconfiguration -- Debug mode, binding to 0.0.0.0, insecure cookies
 - * A06 Vulnerable Components -- (detected by LLM, not pattern scanner)
 - * A07 Authentication Failures -- Plaintext password comparison, no token verification
 - * A08 Data Integrity Failures -- Insecure deserialization (pickle, `yaml.load`)
 - * A09 Logging Failures -- (detected by LLM contextual analysis)
 - * A10 SSRF -- (detected by LLM contextual analysis)
-

Q: "What's the most technically challenging part of this project?"

A: The **LLM integration reliability**. The challenge isn't making one LLM call -- it's making the system work reliably across:

- * Different LLM providers (Ollama, OpenAI, Groq, llama.cpp)
- * Inconsistent JSON output formats
- * Timeout handling (some calls take 60+ seconds)
- * Token limits (chunking large codebases)
- * Graceful degradation when LLM is unavailable
- * Deduplication between pattern and LLM findings

I solved this with a factory pattern for LLM clients, a robust JSON parser, hard timeouts via thread pools, chunking with character limits, and the pattern-based fallback layer.

Q: "What design patterns did you use?"

A:

- * Factory Pattern -- `get_llm()` returns different LLM clients based on config
- * State Machine Pattern -- LangGraph StateGraph for workflow orchestration
- * Strategy Pattern -- Pattern-based scan vs. LLM-based scan (two strategies for analysis)

- * Observer Pattern -- Status updates via state field, polled by frontend
 - * Chain of Responsibility -- Agents process sequentially, each building on previous output
 - * Fallback/Graceful Degradation -- Pattern scanner always works; LLM is enhancement
-

Q: "Why Pydantic for data models?"

A:

- * Validation at boundaries: When LLM returns JSON, Pydantic validates it matches the expected schema. Invalid fields are caught immediately.
 - * Serialization: `.model_dump()` converts models to dictionaries for JSON API responses.
 - * Documentation: Field types serve as living documentation -- anyone reading `SecurityFinding` knows exactly what fields exist and their types.
 - * FastAPI integration: FastAPI uses Pydantic natively for request/response validation.
-

Q: "How does the frontend show real-time progress?"

A: Streamlit polls the `/api/audit/{id}/status` endpoint every 2 seconds. Each agent updates state["status"]` to one of: `queued`, `scanning`, `auditing`, `reviewing`, `generating`, `complete`. The frontend maps these to a 5-step progress bar with visual indicators ([v] done, (time) current, ? pending).`

Q: "What would you add in v2?"

A:

- * Tree-sitter AST parsing -- Multi-language AST support beyond Python
 - * GitHub Actions integration -- Run audit on every PR
 - * PR Comment Bot -- Post findings directly on pull requests
 - * Historical tracking -- Compare audit scores over time
 - * Custom rules -- Let teams define their own patterns
 - * SARIF output -- Standard format for IDE integration
 - * WebSocket -- Replace polling with real-time push updates
 - * Multi-language prompts -- Optimize LLM prompts per language
-

19. Future Enhancements

- * Tree-sitter AST parsing for multi-language support (JavaScript, TypeScript, Java, Go)
- * GitHub Actions integration for CI/CD pipeline automated audits
- * PR comment bot that posts findings directly on pull requests
- * Historical trend tracking across multiple audits of the same repo
- * Custom rule configuration -- YAML-based custom patterns
- * SARIF output format for IDE integration (VS Code, IntelliJ)
- * WebSocket-based progress instead of polling
- * Caching layer to avoid re-analyzing unchanged files

- * Multi-tenant support with user authentication and audit history
- * Dependency vulnerability scanning using OSV or Snyk APIs

20. Key Terminology Glossary

Term	Meaning
LangGraph	LangChain library for building agent workflows as stat..
StateGraph	A directed graph where nodes are functions and edges d..
Conditional Edge	An edge that routes to different nodes based on a deci..
OWASP Top 10	Standard list of the 10 most critical web application ..
SQL Injection	Attacker inserts SQL code via user input to manipulate..
XSS	Cross-Site Scripting -- injecting malicious scripts in..
Pydantic	Python library for data validation and serialization u..
FastAPI	Modern Python web framework with async support and aut..
Streamlit	Python library for building interactive web apps with ..
Ollama	Platform for running LLMs locally on your machine
AST	Abstract Syntax Tree -- structured representation of s..
Health Score	0-100 score representing overall code quality and secu..
Pattern Scanner	Regex-based analysis engine that works without any LLM
Re-Audit	Deeper focused analysis triggered when too many critic..
UUID	Universally Unique Identifier -- used for tracking ind..
CORS	Cross-Origin Resource Sharing -- HTTP header for contr..
Groq	Cloud inference provider offering fast LLM API (OpenAI..
Render	Cloud platform for deploying web applications (used fo..

Quick Summary Card (for last-minute revision)

PROJECT: AI Multi-Agent Code Review & Security Audit System

WHAT: 4 AI agents analyze codebase for security vulns + code quality
HOW: LangGraph state machine -> Scanner -> Security -> Quality -> Report
WHY: Automates manual code review, detects OWASP Top 10, gives health score
STACK: FastAPI + LangGraph + Ollama/OpenAI + Streamlit + Pydantic + Plotly
KEY: Conditional re-audit edge, pattern fallback, robust LLM parsing
INPUT: ZIP upload or GitHub URL
OUTPUT: Health score (0-100), severity chart, findings with fix suggestions

AGENTS:

1. Scanner -- Traverse files, extract code, detect entry points
2. Security -- OWASP Top 10 detection (patterns + LLM)
3. Quality -- Anti-patterns, error handling, AST analysis
4. Report -- Health score, executive summary, compile all findings

DIFFERENTIATORS:

- LangGraph (not CrewAI) for true state machine with conditional edges
- Works without LLM (40+ regex patterns as fallback)
- Robust JSON parsing (4-stage pipeline for LLM output)
- Real-time progress tracking (status polling)
- Configurable LLM (Ollama/OpenAI/Groq/llama.cpp)